Empirical Software Engineering manuscript No.
(will be inserted by the editor)

When Compression Backfires: An Online Dose–Response Audit of Three Lossless Context-Compression Mechanisms for Coding Agents

[AUTHORS TO BE PROVIDED BY THE RESEARCH LEAD]

Received: date / Accepted: date

Abstract Lossless compression of tool outputs is widely assumed to reduce the token cost of LLM coding agents. Whether removed bytes actually reduce what the model consumes — the only outcome that matters for cost — is rarely tested online. We present a paired, pre-registered online audit of three lossless compression mechanisms — repeated-delivery folding, eligibility-gated folding, and side-store externalization with on-demand recovery — across 168 disjoint tasks and 404 real agent runs of one stack (opencode; GLM-5.3-Flash). All three mechanisms preserve task correctness, yet none yields net input-token savings: folding 561K characters of repeated output *increased* non-cached input tokens by 7.1%; safety gating eliminates the harm only by eliminating the mechanism (2 fold events in 30 pairs; an offline grid over 64 eligibility configurations shows no configuration combines $\geq$2% safe exposure with $\geq$1/3 case coverage); and side-store externalization *increases* input tokens by 23.0% (bootstrap 95% CI $[-40.8, -7.6]$) because each on-demand recovery re-sends the session context and amplifies interaction turns (+4.7 tool calls, +50.5K flow tokens per case). We dissociate compression opportunity, triggerable exposure, and realized benefit; attribute the inversion to compensation, recovery demand, and turn-amplified cache traffic; and show that the safe compressible surface ($\approx$1.7% of context) sits below the task-level noise floor ($\hat{\sigma}_w \approx 0.155$, CI $[0.093, 0.197]$). A replay-verified diagnostic audit further separates byte-level losslessness (intact) from immediate in-context diagnostic visibility (incomplete in the deployed mechanism). We release all protocols, 424 trajectories, the census toolkit, and the audit chain.

Keywords coding agents · context compression · empirical study · token economics · negative results

1 Introduction

Coding agents accumulate context rapidly: every tool call returns bytes that enter the conversation and are re-sent to the model on every subsequent turn. Because trajectories are dominated by large tool outputs — file views, test runs, build logs, search results — lossless compression of this traffic is an attractive lever on token cost, and a growing body of work compacts, summarizes, or externalizes agent context Xiao et al. (2025); Zeng et al. (2026); Kang et al. (2026); Kovacs (2026); Ren et al. (2026). Between an offline compression ratio and an online bill sits a chain of confounds: the model may re-read what was hidden (compensation), the eligibility rules that make compression safe may shrink the compressible surface to nothing (exposure collapse), and any recovery path re-introduces content *plus* an extra interaction turn (turn amplification).

This paper reports a complete, pre-registered evidence chain that traverses this gap on a real agent stack. We begin from an opportunity census over 2.76 billion observation characters (20,000 trajectories) showing that a reversible folding candidate (RDS) could address up to 95.96% of opportunity mass — the largest

Autonomous experimental campaign (TASCO-R4/R5/R6), human-supervised. All artifacts, pre-registrations, and full trajectories are hash-frozen and listed in the artifact index. This version uses only frozen evidence and adds no new online experiment. Author information: to be provided by the research lead (see title page).

[AFFILIATION AND CORRESPONDING-AUTHOR E-MAIL TO BE PROVIDED BY THE RESEARCH LEAD. If no institutional affiliation exists, EMSE rules apply: city and country of residence only.]

compressible surface we could construct TASCO Research Group (2026). We then measure, in paired online experiments, what actually happens when that candidate — and two successor mechanisms designed to repair its failure modes — are deployed on 168 disjoint tasks.

Contributions. C1: Opportunity ≠ triggerable exposure ≠ realized benefit. A mechanism-by-mechanism dissociation on real workloads: offline opportunity mass (561K–2.8M characters) is far larger than what the safety contract actually lets the mechanisms trigger online (2 fold events under gating; 280 stored views), and what triggers then *loses* 7.1–23.0% net input tokens while preserving task correctness (Section 5.1), with per-family structure and outliers reported in full (Section 5.2). **C2: An empirical account of benefit and its cost compensation in real agent execution.** Three measured compensation channels invert the character savings — reacquisition and re-exploration (more turns under folding), recovery demand (43.4% of externalized views are recovered, with no pre-observable separating feature), and turn-amplified cache traffic (+50.5K flow tokens per case; the *turn*, not the re-injected bytes, carries the cost) (Sections 5.4 and 5.5). **C3: An auditable evaluation methodology, and a diagnostic-evidence risk result.** A pre-registered, claim-audited evaluation protocol for compression mechanisms — paired arms, frozen oracles, dual input calibers, an offline exposure/eligibility grid, a recovery screen, and a byte-exact replay audit — together with two substantive results: the safe compressible surface is ≈1.7% of context against a task-level noise floor of $\hat{\sigma}_w \approx 0.155$ (CI $[0.093, 0.197]$), and keyword-caliber diagnostic audits can pass thousands of views while semantic diagnostic lines are lost — byte-level losslessness and immediate diagnostic visibility are distinct properties (Sections 5.3, 5.5, and 5.6). A companion measurement study characterizes the opportunity surface itself (taxonomy, cross-corpus replication, capability coverage) TASCO Research Group (2026); this paper is its online counterpart and can be read independently.

2 Background and Related Work

Agent context economics. LLM agents re-send their conversation on every turn, so a tool output delivered at turn k costs its tokens once at delivery and once per subsequent turn (amortized by prompt caching). Context-compression proposals range from summarization and truncation to external memory stores; most are evaluated by offline compression ratios or downstream task scores. The *token economics* of the mechanism itself — what the deployed compressor does to actual billed input — is rarely the dependent variable.

Trajectory-compression mechanisms. AgentDiet manually classifies waste in 100 Trae-Agent trajectories and reduces trajectory cost Xiao et al. (2025) — the closest prior system study. AttnCompress compresses trajectories under attention guidance Zeng et al. (2026); Paritok-4B trains an intent-conditioned extractive compressor over 67K OpenHands trajectories Shi and Chen (2026); SWE-Pruner prunes task-relevant code context Wang et al. (2026b); TACO self-evolves terminal-observation compression rules Ren et al. (2026); Squeez learns minimal verbatim evidence retention for tool outputs Kovacs (2026); CORVUS targets file-state synchronization to prevent stale rereads Zheng et al. (2026); ACON optimizes context compression for long-horizon agents Kang et al. (2026); and CoACT trains an action-preserving compressor whose compressed observations must induce the same next action as the raw ones Chen et al. (2026). ReSum and AgentFold apply periodic context summarization to web agents Wu et al. (2025); Ye et al. (2025). ICLR-style reasoning-block elision and agentic context management systems extend the family Wang et al. (2026a); Li et al. (2026); He et al. (2026), and TRACE studies verifier-guided compaction Min et al. (2026). All of these answer *how to compress*; none reports paired online token economics of a lossless mechanism across its full deployment cycle, which is our object.

Cost and fidelity of compression. Closest to our measurement, an empirical cost attribution decomposes gateway-level compression costs Chen and Shi (2026); a fidelity study shows LLM summaries can distort decisions Lee et al. (2026), motivating preservation gates. Neither measures end-to-end paired input-token outcomes of a frozen lossless mechanism, and neither connects behavioral compensation to the loss.

Evaluation methodology for agents. A 60K-trajectory stochasticity study recommends multi-run evaluation with power analysis for sparse-activation mechanisms Bjarnason et al. (2026), corroborating our noise-floor result; process-centric and failure anatomy studies characterize behavior but not compression economics Liu et al. (2025); Zhao et al. (2026); harness-design ablations compare five context-management strategies Fan et al. (2026); thought-action-result analyses Bouzenia and Pradel (2025) and trajectory fingerprinting Oderinwale (2026) complete the behavioral picture; context-rot effects bound long-context skills Xue (2026). Benchmarks and platforms (SWE-bench, SWE-agent, OpenHands) evaluate task correctness Jimenez et al. (2024); Yang et al. (2024); Wang et al.

(2024); we use the same stack style but measure *cost*, with frozen oracles.

Execution-effect and policy studies. Closest to our evaluation question, three recent studies measure what compression does to the *executing* agent rather than to single prompts. A systematic policy study varies what, when, and how much to compress across three open-weight models on SWE-bench Verified and Terminal-Bench (≈35K runs) and finds that fewer tokens need not mean faster or cheaper execution — policies using roughly one-third as many tokens can take 20–80% longer than the uncompressed agent, with strong model and workload dependence Satish et al. (2026). A controlled reacquisition-cost protocol shows that compression can leave task completion statistically unchanged while retrieval calls surge (21.0 → 63.9 calls, p=.002, with completion 80%→85%), and that the signature is environment-dependent Liu (2026). And at an equal token budget, selecting whole files verbatim beats compressors and retrievers on SWE-rebench issue resolution — partly because compressors alter the text that patches must reproduce Yang et al. (2026). A public benchmark (not peer-reviewed) holds a fixed agent, model, and the official SWE-bench grader constant and ranks *shipped* compression products by cache-aware cost per solved task, reporting layers on both sides of the no-compression baseline Dasein Labs (2026) — its cache-aware accounting reinforces the point that visible-token reductions do not translate into dollar savings when the cached prefix churns. These studies evaluate harness-level compression *policies* or shipped products; none freezes a specific lossless, recoverable mechanism with byte-exact recovery, pairs it against a baseline under frozen oracles, and audits what the deployed view actually retained.

Position. We do not claim to be the first to evaluate compression end-to-end; the studies above already measure execution effects at increasing breadth. Our contribution is complementary and specific: a *pre-registered, paired, task-oracled* audit of three *frozen, lossless, recoverable* mechanisms on real repository tasks, with the failure attributed mechanism by mechanism (compensation, exposure collapse, recovery re-injection), the safe compressible surface measured against the task-level noise floor, and a byte-exact replay audit separating byte-level losslessness from immediate diagnostic visibility. Positive-savings mechanisms (AgentDiet, CoACT, ACON) remove or semantically rewrite content without byte-exact recovery obligations; our results bound the complementary regime — deterministic, recoverable externalization — where the measured outcome is a robust loss.

3 Study design and mechanisms

All three mechanisms wrap the same frozen primitive family and share the deployment contract: anchor-first delivery, exact-content-only folding, fail-closed behavior on any inconsistency, auditable markers, and byte-exact reconstruction (Figure 1). Losslessness here means byte-exact reconstruction from the side store or from re-delivered anchors; it does not by itself guarantee that every diagnostically relevant line is *directly visible* in context — the two properties are distinct (Section 5.6).

M1 — Repeated-delivery folding (RDS, high dose). Any contiguous run of ≥2 lines exactly matching a contiguous run delivered in a *single earlier* observation is replaced by an audit marker; the first delivery is always verbatim (anchor-first). Deployed at full coverage in R4: every tool output in every run.

M2 — Eligibility-gated folding (safe dose). The same primitive with three gates: diagnostic-bearing outputs are never folded; target/specification file reads are never folded; read outputs fold only from the third delivery of the same path; non-read outputs fold only if ≥1,000 characters with duplicate-line share ≥0.30. Gates use only runtime-observable facts.

M3 — Side-store externalization (LargeView). Read outputs ≥4,000 characters are replaced in-context by head (20) and tail (20) windows, verbatim copies of lines matching a frozen eight-keyword diagnostic pattern (`fail|error|assert|traceback|exception|fatal|denied|not found`), and a pointer marker; the full original is written losslessly to `.r5-store/<sha16>.txt` for recovery by an ordinary read. *Post-hoc disclosure:* this keyword caliber does not retain failure returns that carry no listed keyword, and markers print raw tool-output line numbers while the view displays content line numbers; both defects are bounded, deterministic, and characterized in Section 5.6.

Each mechanism is deterministic, requires no model calls, and was frozen (code hash, protocol, oracle set) before its online evaluation.

4 Methodology

Frame and stack. Four experiments over 168 disjoint tasks drawn from two real JavaScript repositories, executed by the opencode harness opencode.ai (2026) driving GLM-5.3-Flash (model identity verified per run from harness logs). Tasks: code review, module analysis, test-fix/TDD, and cross-specification analysis — selected by frozen seeded rules from unused material;

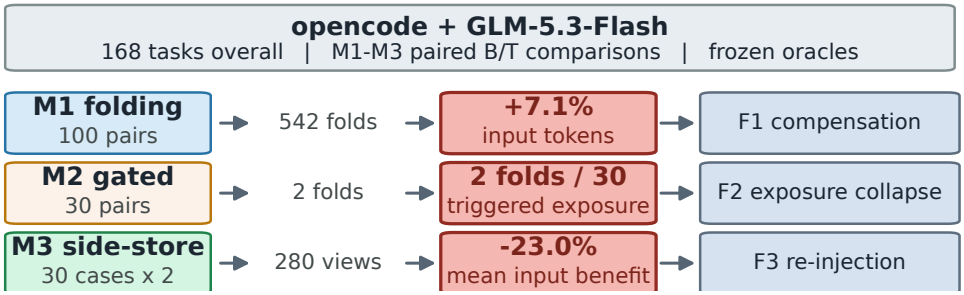


Fig. 1 Study overview and headline outcomes. Three lossless-recoverable compression mechanisms (lanes) were audited in paired online runs on one stack. Badges show each mechanism's primary observed result on its own metric: M1, input-token consumption increase (+7.1%, totals caliber); M2, triggered fold exposure (2 of 30 pairs); M3, mean input-token benefit (−23.0%, case-equal-weight, 95% CI [−40.8, −7.6]; Table 2). F1–F3 are mechanism interpretations supported by the execution trajectories and are not independently proven causal conclusions. Metric definitions, denominators, and applicable confidence intervals are given in Section 4 and Table 2.

no task was chosen or dropped after observing results. Case-level disjointness across the four experiments is verified programmatically (zero pairwise case-ID intersections); within M3, the two repetitions per arm are repeated measures on the same cases, aggregated to arm means *before* pairing (60 runs are not 60 independent tasks). The LLM under study serves as the execution engine of the audited stack; it is not an author of this work, and its use is documented here and per run in the harness logs.

Design. Paired per case: every case executes once with compression disabled (Baseline, B) and once enabled (Treatment, T), in seed-balanced AB/BA order, with a fresh copy of the pinned repository per run. M3 adds a second repetition per arm (2 reps × 2 arms). A baseline-only noise pilot (Stage-0) measured within-task variance on 8 tasks × 3 repetitions.

Calibers (pre-registered). Table 1. *Input tokens* (non-cached, api-reported; primary), *cache-read tokens*, *output tokens*, *flow tokens* (sum of the former three plus reasoning), *wall time*, *tool calls*, *oracle success* (frozen test commands or deterministic text-token oracles), *fold/store events*, and *recovery reads*. Character counts are opportunity proxies only and are never added to token quantities.

Statistics. Case-level pairing. We report two pre-declared input aggregates and never mix them: a *totals-based* delta, $(\sum T - \sum B)/\sum B$, which weights cases by their size (the historical headline caliber for M1), and a *case-equal-weight* mean of per-case relative deltas with case-level bootstrap 95% CIs (10,000 resamples; the headline caliber for M3). Robustness via medians, win/loss/tie counts, and leave-one-out sensitivity. Families with fewer than 10 cases are marked *exploratory*. Uncertainty via case-level bootstrap; the Stage-0 noise floor is $\hat{\sigma}_w = 0.155$ (task-level bootstrap 95% CI $[0.093, 0.197]$), and the expansion directly measured $\hat{\sigma}_w(T) = 0.258$ vs. $\hat{\sigma}_w(B) = 0.244$ (consistent with equality at this sample size — not a proof). A pre-registered 15-claim audit with evidence grades (Section 7) governs what each result may be used to claim.

Table 1 Measurement calibers and their cautions.

Caliber	Definition	Caution
Chars saved	obs chars removed by fold/store	proxy; not tokens
Input	non-cached input tokens (api-reported)	primary metric
cache_read	cached re-send tokens	amplifies with turns
Flow	in+out+reasoning+cache	session-length sensitive
Success	oracle passed	B/T same oracle

Offline instruments. Three deterministic, zero-model-call instruments close the design: (i) an eligibility *grid* that replays all 100 baseline trajectories under 64 gate configurations; (ii) a *recovery screen* that censuses every externalized view for recoveries, distances, and per-view character nets; (iii) a *replay audit* that reconstructs every compressed view byte-for-byte from frozen artifacts (used by the Section 5.6 case study).

5 Results

5.1 RQ1: Does lossless compression reduce real input tokens? No.

Table 2 unifies the three mechanisms. Figure 2 shows the primary metric; Figure 3 the per-case distributions.

Table 2 Unified mechanism table (columns = mechanisms; calibers never mixed). "Input Δ (totals)" weights cases by size; "benefit (case mean)" is the case-equal-weight mean of per-case relative deltas with bootstrap 95% CI. Benefit values are color-coded: red = input loss, green = saving.

Metric	**M1 folding** *100 pairs, 1 rep/arm*	**M2 gated** *30 pairs, 1 rep/arm*	**M3 side-store** *30 cases, 2 reps/arm*
Online exposure	542 folds; 561K chars; 64/100 cases	2 fold events; 2/30 cases	280 views; 2.81M chars; 123 rec.
Input Δ (totals)	+7.1%	−2.9%	+12.4%
Benefit, case mean [95% CI]	−41.8% [−66.7, −20.2]	−21.3% [−50.3, +2.6][a]	−23.0% [−40.8, −7.6]
Median benefit	−4.0%	+4.9%	−15.3%
Cache B→T	10.27→10.74M (+4.6%)	5.36→4.42M (−17.6%)	2.20→3.58M (+63.2%)
Turns / tool calls	+50 harness steps (total)	tool-call ratio 1.02	+4.7 calls; +2.4 turns
Task B→T (oracle)	81→80	24→24	18→20
Notable case	C-ENT04 (T input 8.7× B)	R-XS-AUTH-REP (−273%)	X-HI-M01 (+23.9%, 0 rec.)

[a] M2's case-mean is dominated by one exploratory outlier; its median is +4.9%, and its paired flow-delta median is −0.25% — the mechanism is near-inert, as the grid predicts. M1's case-mean is outlier-heavy too; medians give the robust picture — still at or below parity. Success rows are observed oracle parity, not a powered non-inferiority test. M3 wall +48%.

M1 (100 pairs): 542 fold events removed 561,454 characters, yet treatment arms consumed **7.1% more** non-cached input than baseline (totals caliber; win/loss on flow 35/62). Task success showed no observed degradation (81% → 80%); this parity is an observation, not a powered non-inferiority test.

M2 (30 pairs): gates eliminated the harm (median paired flow delta −0.25%, tool-call ratio 1.02, success 24/24 both arms) — but also eliminated the mechanism: only 2 fold events fired across 30 cases.

M3 (30 cases × 2 reps): 280 stored views (2.81M characters externalized), 123 recovery reads; treatment arms consumed **23.0% more** non-cached input (case-equal-weight; bootstrap 95% CI [−40.8%, −7.6%]; median −15.3%; win/loss 8/22). Success 18/30 → 20/30. Across 168 disjoint tasks and three designs, no configuration produced positive net input savings, and the two mechanisms with measurable exposure produced losses whose bootstrap 95% CI excludes zero (Figure 2).

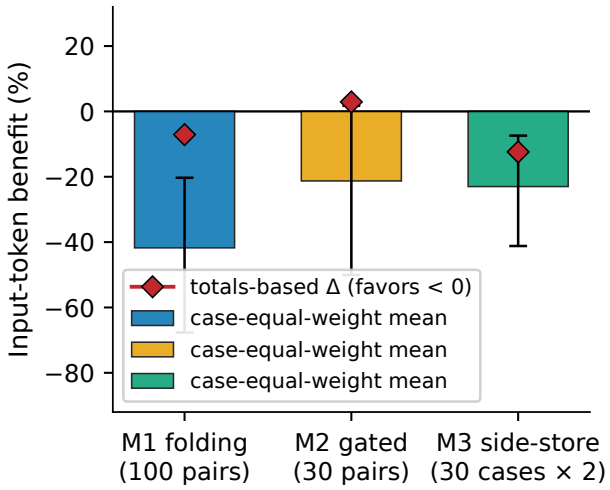


Fig. 2 Primary metric by mechanism: case-equal-weight mean input benefit with bootstrap 95% CI (bars), and totals-based input delta (diamonds; positive value = treatment used more input). Zero line = parity. All three mechanisms are at or below parity.

5.2 Family structure and robustness

Table 3 breaks the primary metric down by task family (Figure 4). Cells with $n < 10$ are exploratory signals, not formal tests; three observations. (i) *No family escapes the negative sign where exposure is measurable*: file-review — the family with the most folding opportunity — is the worst under M1 (−59.9% mean, CI excluding zero), exactly where compensation predicts the largest harm. (ii) *Positive cells are small, exploratory, and mechanism-consistent with near-zero exposure*: M1 spec/contract (+7.1%, n=10, CI includes zero) and M3 cross-spec (+30.3%, n=2) have the least exposure. (iii) *Oracle parity hides family-level movement*: M1 spec/contract success moves 4→2/10 and M2 mutation 8→7 — single-family counts too small for inference, reported for completeness. Robustness: M3's headline survives leave-one-out (−17.2pp without the largest outlier, X-NO-F01; median −15.3%; win/loss 8/22), and a zero-exposure control subgroup (6 cases with no markers; mean +10.6%, median +2.4%) is statistically indistinguishable from zero, excluding systematic T-arm penalties such as plugin loading or ordering effects. M1/M2 means are outlier-sensitive (leave-one-out ranges in the supplement); their medians (−4.0% / +4.9%) are the robust summary — still at or below parity.

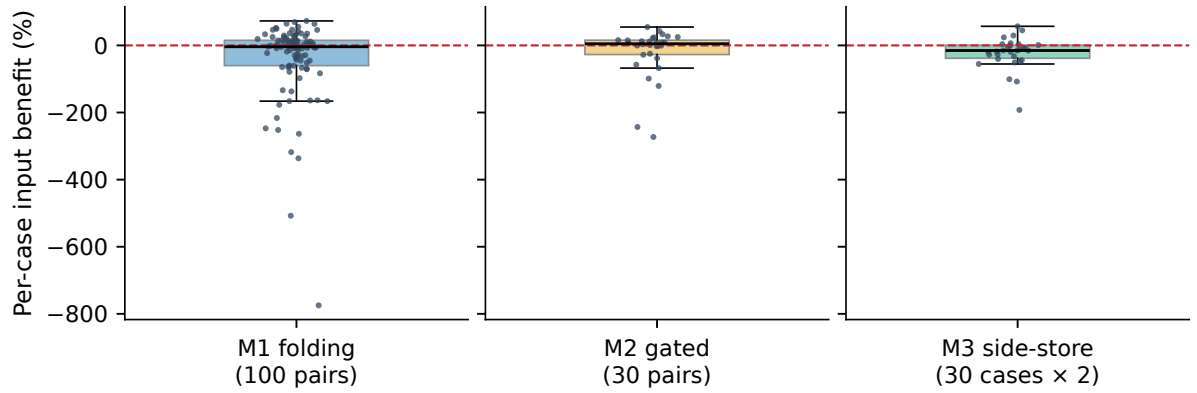


Fig. 3 Per-case paired input benefit distributions (points jittered; box = quartiles; zero = parity). Distributions are wide relative to their means — the noise floor of Section 5.3.

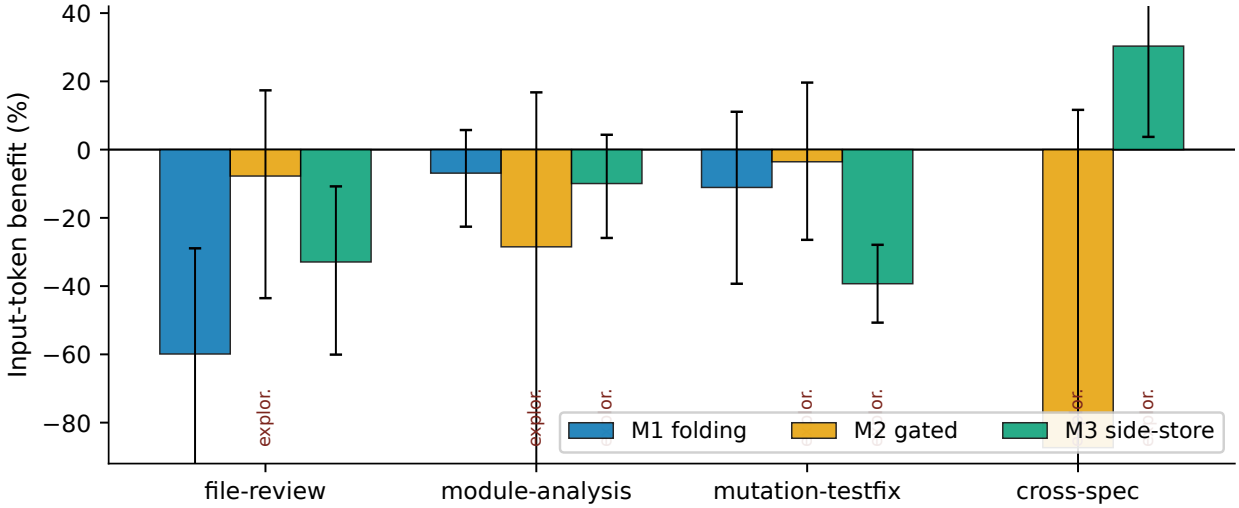


Fig. 4 Per-family input benefit (mean, bootstrap 95% CI) by mechanism. Exploratory cells ($n < 10$) are marked. No family with measurable exposure escapes the negative sign.

Table 3 Per-family input benefit (case-equal-weight mean, 95% CI, win/loss) by mechanism. $n < 10$ = exploratory. Families are separate tasks, not replications.

Family	Mech	n	Benefit [CI] (%)	W/L
file-review	M1	48	−59.9 [−95.3, −29.0]	17/31
	M2	8[e]	−7.7 [−43.4, +17.2]	4/4
	M3	18	−32.9	
module-analysis	M1	24	−6.9 [−22.2, +6.1]	12/12
	M2	8[e]	−28.5 [−96.5, +16.9]	5/3
	M3	8[e]	−9.9	
mutation-testfix	M1	15	−11.1 [−38.8, +11.0]	6/9
	M2	8[e]	−3.6 [−26.5, +19.6]	5/3
	M3	2[e]	−39.3	
spec/contract	M1	10	+7.1 [−21.2, +33.0]	7/3
cross-spec	M2	3[e]	−87.3 [−272.9, +11.7]	1/2
	M3	2[e]	+30.3	
enterprise	M1	3[e]	−346.7 [−774.7, −18.4]	0/3

5.3 RQ2: How large is the *safe* compressible surface? Below measurability.

The exposure–safety tension is not a tuning artifact. An offline grid replayed all 100 baseline trajectories under 64 eligibility configurations (read-delivery threshold × duplicate-share threshold × minimum size; 70/30 calibration/holdout split). **No configuration** satisfies the pre-registered acceptance gates of ≥1/3 case coverage and ≥2% safe exposure (Figure 5): the Pareto frontier tops out at **1.66%** exposure (the most permissive safe configuration, which is exactly the M1 harm regime minus protected content), and the configuration matching M2's gates replays to **zero** fold events on the same trajectories while its protection gates block 245 target/specification/diagnostic folds — explaining M2's near-inertness directly. The safe surface is structurally below the task-level noise floor: at $\hat{\sigma}_w \approx 0.155$, no repli-

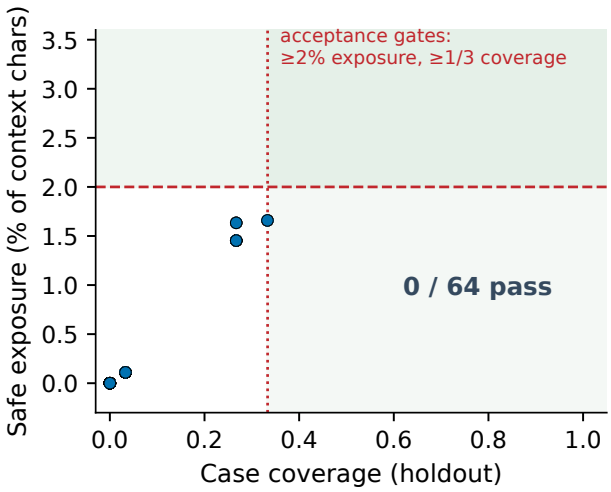


Fig. 5 Offline eligibility grid (64 configurations, holdout replay): safe exposure vs. case coverage. Dashed/dotted lines: pre-registered acceptance gates (≥2% exposure, ≥1/3 coverage). No configuration passes; the frontier tops out at 1.66%.

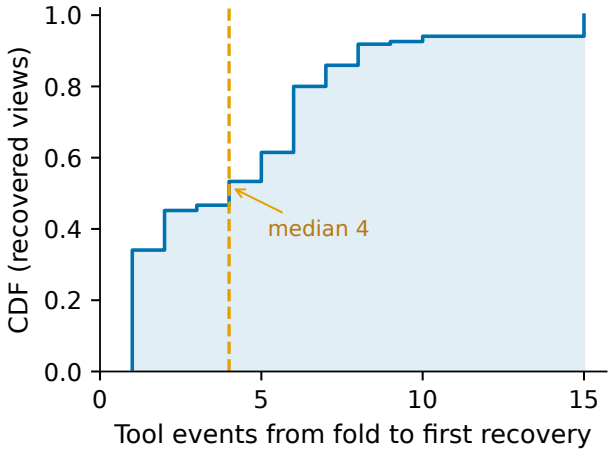


Fig. 6 CDF of the event distance from a fold to its first recovery (recovered views, n=135; median 4). Recoveries come early, re-pricing the session while its context is largest.

cation count ≤10/arm reaches 0.80 detection probability for effects of +3–5% Bjarnason et al. (2026).

Cross-corpus exposure. A second-workload census (15,000 sessions from an independent corpus) confirms that the *exposure* pattern generalizes: side-store eligible views trigger 52,719 times, saving 366M characters — 29.5% of all observation characters there. Exposure is not the bottleneck on that corpus; its *recovery economics* remain untested online. Both instruments are character-level opportunity proxies on recorded data — they cannot establish online token benefit, which is precisely the lesson of this paper.

5.4 RQ3: Can eligibility features predict recovery need in advance? No.

Across 311 externalized views (70 treatment runs), 135 (43.4%) were recovered at least once (137 recovery events; median distance 4 tool events after the fold; Figure 6). Recovery rates by size bucket, file extension, task family, and exposure stratum all fall in the 38–52% band with no separating feature, in both the design split and a 30% holdout (43.4% vs. 49.3%). Per-view character nets remain positive even including recoveries (design +4,241, holdout +4,589 chars/view) — yet the same cases lose 23% input tokens. The only strongly positive exposed case (X-HI-M01: +23.9%) had **zero** recoveries — the winning condition ("the model never needs the content back") is observable only post hoc. A conservative caveat: 24 same-file rereads occurred in baseline arms without any side store, so "no recovery" does not mean "no need"; needs can resurface as re-derivation, not reads.

5.5 Why character savings become token losses: turn amplification

The central measurement lesson is that *character accounting and API token accounting are separate calibers, and their signs diverge.* We therefore report them separately and never add them.

Caliber A — API tokens (primary, api-reported). Per case, M3 treatment arms consumed +2,577 more non-cached input tokens, +46,265 more cache-read tokens (+50.5K flow tokens), issued +4.7 tool calls and +2.4 model turns, and took 48% longer. The dominant observed cost is cache-read traffic, which scales with session length and turn count: Δcache tracks Δflow almost perfectly (r=0.997), and Δcache correlates with Δinput at r=0.33.

Caliber B — mechanism characters (measured, arm means). The primitive saved S = 650,693 characters, re-injected R = 158,880 through recovery reads, and added M = 40,600 marker characters: a net *character* saving of +451,214. The per-case decomposition leaves a residual X = 528,537 characters that absorbs the unit conversion and treatment-arm behavioral divergence (output +13%, wall +48%); we do not interpret X causally. *Caliber note:* recovery re-injection also exists in a per-run caliber (352,804 characters over all 70 treatment runs including smoke runs, or 317,759 over the 60 expansion runs); the arm-mean caliber above is the one compatible with per-case pairing. Both are frozen and reconciled in the artifacts.

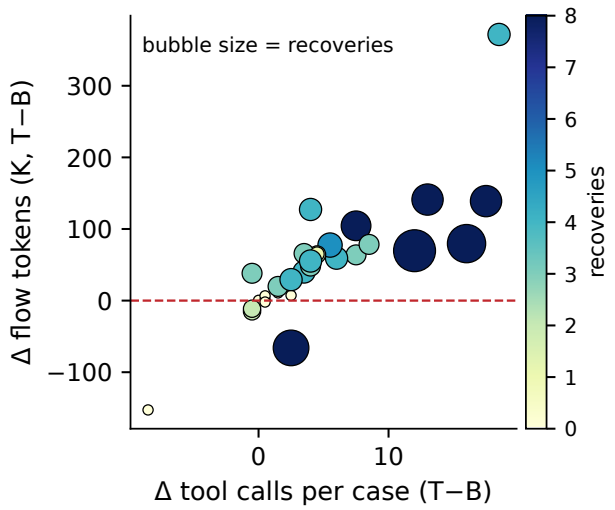


Fig. 7 Turn amplification: Δ tool calls vs. Δ flow tokens (K) per case; bubble size = recoveries. More recoveries ⇒ more turns ⇒ more re-sent context.

The divergence. Why does a +451K-character saving coexist with worse API-token outcomes? Because the binding cost is not the returned bytes but the *turn*: each recovery is mechanically one additional tool call whose re-sent context is billed largely as cache reads. Consistently, the *size* of what comes back barely predicts the loss (re-injected chars vs. per-case input delta: r=0.03), while *how much extra traffic the session generates* does ($r(\Delta\text{cache}, \Delta\text{input}) = 0.33$). Treatment trajectories also diverge behaviorally (exploration, output +13%), which we measure but cannot causally decompose without counterfactual arms; we therefore claim only the association, in both directions. Per-view character accounting is structurally blind to this effect; any compressor with a recovery path is exposed to it (Figure 7).

5.6 A bounded diagnostic-visibility case study: the post-hoc G5 audit

Motivation and method. The original claim that M3 retains "all diagnostic lines verbatim" had been verified only in a *keyword caliber*: a frozen audit checked that keyword-matching line counts are identical before and after folding across 24,648 replayed views. A post-hoc audit (G5) tested the *semantic* caliber on 30 blind views drawn from 3 evidence runs (10/13/7 views; 3 underlying cases). Two isolated LLM reviewers reviewed the blind package under a frozen protocol; every verdict-driving claim was then re-verified mechanically, so the findings below do not rest on model votes. *This is a model-assisted audit; independent human review remains pending.*

Findings (all mechanically verified by deterministic replay). (1) *E1, semantic diagnostic loss from the view*: 12 unique failure-return lines across the 3 target files (4 per file, `return { ok: false, reason: "exhausted_retries",attempts: attempt };`), appear in none of the 30 in-context views; they are retained only in the byte-exact side store. The deployed keyword pattern does not match them. The store itself is faithful: stored content equals the repository files line-for-line (30/30), marker hashes match, and each in-context view is byte-identical to the deterministic replay of the frozen folding function over the stored original (30/30) — the view is fully explained by the mechanism, and the agent session streams contain exactly the folded views. (2) *E3, marker coordinate inconsistency*: markers declare the omitted range in *raw tool-output* line numbers (`21..198/199`) while the surrounding view displays *content* line numbers; the 3-line read envelope shifts the two systems by a constant, so a reader reconciling marker and view concludes that content lines 18–20 are unaccounted. In content coordinates the declared range exactly abuts the head and tail windows: no content is undeclared-omitted; the defect is one of auditability, not of content loss.

Scope and status. The audit covers 3 evidence runs (3 cases, 30 views); it does not estimate a population defect rate, and the 30 views are not 30 independent tasks. The two findings are deterministic properties of the frozen plugin (one keyword-list coverage gap; one coordinate-system choice), not random failures. The economic results above are measured in token calibers and are unaffected; oracle correctness is unaffected. The claim "all diagnostic lines verbatim" is **withdrawn** in favor of the keyword-caliber statement in Section 3 plus this case study; byte-level losslessness and recovery integrity were independently re-verified and stand. A repair design (semantic diagnostic protection with structure-aware failure-return detection and coordinate-consistent markers) is released with the artifacts; a repaired mechanism is a *new* mechanism and would require fresh online measurement.

6 Discussion: a reusable failure model

The three mechanisms fail in three distinct, now-measured ways, and the failures generalize into a pre-deployment checklist for any context compressor:

F1 — Compensation (M1). Folding hides content the model was using: harmed families explore *more* (file-review: +24% total harness steps under M1, computed from frozen per-case data), consistent with agents

re-acquiring hidden context. *Gate: measure compensation before deployment; folding is not free just because it is reversible.*

F2 — Exposure collapse (M2). Safety gates remove exactly the folds that caused F1 — and with them 98% of the exposure; the grid shows no parameter point escapes the frontier. *Gate: run the exposure census first — if the safe surface is below a few percent of context, the mechanism cannot produce a measurable effect.*

F3 — Recovery re-injection (M3). Side-store recovery restores full content natively plus one extra turn; recoveries come early (median 4 events), repricing the session when its context is largest, and no pre-observable feature separates low-recovery views. *Gate: price recovery as turns, not bytes; require a turn-neutrality argument for any call-back path.*

For practitioners. (1) Run the exposure census — opportunity proxies are necessary but not sufficient; (2) price recovery in turns and cache traffic, not characters; (3) measure per-turn amplification, not per-fold savings; (4) verify correctness with the same oracle as the deployment target; (5) verify *diagnostic visibility* against a semantic definition of "diagnostic," not a keyword list — our keyword-caliber audit passed 24,648 views while missing every non-keyword failure return (Section 5.6).

For researchers. The exposure–measurability gap generalizes the variance-dominance problem of agent evaluation: mechanism exposure must exceed the within-task noise floor ($\hat{\sigma}_w \approx 0.155$ here) by the detection margin, or online experiments are underpowered regardless of sample size. Our census toolkit computes both quantities offline.

Applicability boundary. The three failure mechanisms were measured on *deterministic, lossless, recoverable* mechanisms on multi-turn repository tasks. They do not extend to the positive-savings regimes reported elsewhere: removal-based reduction Xiao et al. (2025), learned action-preserving compression Chen et al. (2026), and guideline-semantic compression Kang et al. (2026) trade byte fidelity (and recovery obligations) for savings, and operate in settings our exposure analysis does not cover. Our compensation findings are nonetheless consistent with independent protocols — reacquisition surges with completion unchanged Liu (2026), and policy-level studies find fewer tokens can mean slower, costlier execution Satish et al. (2026) — suggesting the compensation channel is not specific to our stack, while the *magnitude* of every number here is.

7 Pre-registered claim audit

Fifteen claims were pre-registered with evidence grades: 7 STRONGLY_SUPPORTED, 4 SUPPORTED_WITH_LIMITATIONS, 2 HYPOTHESIS_ONLY, and 2 UNSUPPORTED (the latter two are overreach formulations we explicitly disavow, including "all compression strategies are useless"). The full table with evidence pointers is in the supplement (Online Resource 1); the load-bearing entries: C1 no net input gain (SS), C2 turn amplification dominates (SS), C3 safe surface ≈1.7% (SS), C4 no correctness regression observed (SS, caliber-limited), C7 offline census predicts exposure (SS), C8 $\hat{\sigma}_w = 0.155$ $[0.093, 0.197]$ (SS), C9 recovery is real agent behavior (SS), C12 turn-neutral silent truncation might work — *only under an explicit diagnostic-visibility guarantee; the deployed M3 lacked one (§5.7)* (H), C13 semantic-fidelity compression viable — *hypothesis only; the deployed keyword-caliber diagnostic protection did not achieve semantic diagnostic visibility (§5.7)* (H).

8 Threats to validity

External validity. One model (GLM-5.3-Flash), one harness (opencode), one language family (JavaScript repositories), and analysis-type tasks dominate the sample; the online evidence base is a single stack. A systematic policy study finds compression effects are strongly model- and workload-dependent Satish et al. (2026), so our numbers should be read within that dependence. The second-workload census is offline (character proxies only), so its recovery economics are untested. Mutation-heavy workloads showed directionally positive subgroups under M1/M3 but are small (n=15, n=8) and were not powered. **Statistical validity.** M3 uses 2 reps/arm aggregated to arm means (30 independent cases); M1/M2 use 1 rep/arm. Case-level bootstrap CIs are reported for all headline means; family cells with $n < 10$ are exploratory. We report both totals-based and case-equal-weight input calibers and do not mix them. **Construct validity.** Oracle success is not diagnostic visibility; the two are kept separate (Section 5.6). Character counts are opportunity proxies. **Measurement and cost.** Harness-side token accounting is verified against api-reported usage, but billing amounts were not observable, and we report no estimated-cost conversions — all cost statements are API-observable metering. A tokenizer-exact, per-recovery causal decomposition is not observable from harness telemetry; the $S/R/M/X$ decomposition is an accounting identity whose residual absorbs the unit conversion, and we do not interpret it as a physical

identity. **Safety-claim scope.** The fail-closed contract is verified by code review, unit and gate tests (17/17 primitive semantics, 9/9 gates), and a zero-incident observation across 424 runs; no systematic fault-injection campaign was executed. Safety claims are therefore limited to these tested invariants and observed behavior, not to arbitrary fault hypotheses. **Internal validity.** B/T trajectories diverge before any marker in companion analyses, so single-run deltas conflate variance with mechanism; the zero-exposure control subgroup and multi-experiment consistency mitigate but do not eliminate this. **Audit and mechanism-verification scope.** The Section 5.6 audit is model-assisted (two isolated LLM reviewers whose verdicts diverged substantially and were therefore not used as evidence; all reported defects verified by deterministic replay). Independent human review is registered but pending; the audit covers 3 evidence runs (3 cases), and only those views are byte-replay-verified — no population defect-rate is inferred, and the remaining runs are covered by the mechanical preflight (direction-inclusive containment) rather than semantic audit.

9 Conclusion

We built the largest safe compressible surface we could construct for a real coding agent, deployed it online under a pre-registered paired design, and measured a robust negative. The measurement chain — offline census, dose–response, gating, recovery accounting, and a replay-verified diagnostic audit — explains why, and the explanation generalizes across the three frozen mechanisms studied here: for lossless compression on agentic workloads, the binding constraint is not compression ratio but the economics of interaction turns and the unpredictability of content re-use. This negative economics is a property of the measured v1 mechanisms; it does not by itself prove that every conceivable repair must lose — a diagnostic-safer externalization (semantic protection with coordinate-consistent markers; repair design released) changes the exposure trade-off and must be judged by the same online measurement protocol before any such claim. We release the census toolkit, all protocols, 424 trajectories, the diagnostic-audit evidence chain, and the repair design to make the next mechanism's evaluation cheaper than ours was.

Declarations

Funding. [TO BE PROVIDED BY THE RESEARCH LEAD — or state "No funding was received for this research."]

Competing interests. [TO BE PROVIDED BY THE RESEARCH LEAD — at minimum disclose that one of the audited compression mechanisms originates from the same research programme as this paper, if applicable.]

Data availability. [TO BE FINALIZED BY THE RESEARCH LEAD. Factual basis: the complete evidence chain — 424 agent-run trajectories, events, protocols, pre-registrations, analysis scripts, and the audit chain — exists and is hash-frozen (artifact index, 1,889 files across R3–R5). Options: (a) public repository DOI of the frozen index; (b) available from the corresponding author on request. Second-corpus census results are aggregate counts over public datasets Thoughtworks (2026); NVIDIA (2026).]

Code availability. [TO BE FINALIZED BY THE RESEARCH LEAD. The analysis scripts (bootstrap, replay audit, census) are deterministic and re-runnable; packaging and release channel per (a)/(b) above.]

AI tools. The studied coding-agent stack executes an LLM (GLM-5.3-Flash served via a Volcano-Engine-compatible gateway opencode.ai (2026)); it is the *object* of measurement, not an author. LLM-based tooling supported parts of the analysis automation; every reported number was verified by deterministic replay over frozen artifacts, and human accountability for the final text rests with the authors.

Scope boundary with the companion study. Paper A TASCO Research Group (2026) measures the compression-*opportunity* surface (taxonomy, replication, capability coverage) on public corpora; this paper reports the *online economics* of three frozen mechanisms on its own paired experiments. All online numbers in this paper come from Paper B's own runs; Paper A's census figures are cited only as prior characterization and are never counted as this paper's experimental results.

A Supplement

The supplement (Online Resource 1, `ESM_1.pdf`) contains: the full caliber table, Stage-0 noise detail, the 64-configuration eligibility grid, robustness detail (win/loss, leave-one-out), the R6 recovery-screening detail, the G5 audit protocol and limited-sample notes, the repair-design summary, the C1–C15 claim table with evidence pointers, and the number-to-evidence map.

References

Bjarnason BH, Silva A, Monperrus M (2026) On the randomness of software engineering agents. arXiv preprint arXiv:260207150

Bouzenia I, Pradel M (2025) Understanding software engineering agents: A study of thought-action-result trajectories. arXiv preprint arXiv:250618824

Chen H, Zhu Y, Zhang Y, Li J (2026) Coact: Action-preserving observation compression for coding agents. arXiv preprint arXiv:260702911 Next-action preservation; SWE-bench Verified; total tokens −33.0%

Chen L, Shi J (2026) An empirical cost attribution of context-compression gateways in multi-turn coding agents. arXiv preprint arXiv:260922114

Dasein Labs (2026) Code-compression bench: A reproducible benchmark of context-compression layers for coding agents on SWE-bench verified. https://github.com/daseinlabs/code-compression-bench, public benchmark artifact, not peer-reviewed; 100 SWE-bench Verified tasks; fixed agent and official grader; cache-aware cost per solved task; results 2026-07-04 and 2026-09-21

Fan RZ, Zhang Z, Ma S, Hu Y, Wang S, Song K, Liu F, Zamani H, Wang X (2026) An empirical study of harness design for coding agents. arXiv preprint arXiv:260920804

He Z, Shen J, Liang C, Liang HN (2026) Hasmem: Hard-origin adaptively softened memory for long-term llm agents. arXiv preprint arXiv:260930797

Jimenez CE, Yang J, Wettig A, Yao S, Pei K, Press O, Narasimhan KR (2024) Swe-bench: Can language models resolve real-world github issues? In: The Twelfth International Conference on Learning Representations (ICLR), arXiv:2310.06770

Kang M, Chen WN, Han D, Inan HA, Wutschitz L, Chen Y, Sim R, Rajmohan S (2026) Acon: Optimizing context compression for long-horizon llm agents. In: Proceedings of the 43rd International Conference on Machine Learning, Proceedings of Machine Learning Research, vol 306, pp 55321–55375

Kovacs A (2026) Squeez: Task-conditioned tool-output pruning for coding agents. arXiv preprint arXiv:260404979

Lee H, Park S, Lee S, et al. (2026) When summaries distort decisions: Information fidelity in llm-compressed financial analysis. In: EMNLP 2026 Industry Track, arXiv:2606.29251

Li X, Ming R, Chu M, Shao S, Jin R, Xiong C (2026) Acm: Agentic context management for long horizon tasks. arXiv preprint arXiv:260723809

Liu S (2026) What does context compression cost an agent? interaction costs unrevealed by task-completion metrics. arXiv preprint arXiv:260816370 Controlled reacquisition-cost protocol; 3 models x 2 regimes

Liu S, Chen Y, Krishna R, Sinha S, Ganhotra J, Jabbarvand R (2025) Process-centric analysis of agentic software systems. arXiv preprint arXiv:251202393

Min G, et al. (2026) Toward reliable context compression for long-horizon agents: An empirical study of execution instability. arXiv preprint arXiv:260806503

NVIDIA (2026) Open-swe-traces. https://huggingface.co/datasets/nvidia/Open-SWE-Traces, v1.0; 151,219 trajectories; 4 scaffold/model strata

Oderinwale H (2026) Agent trajectories as programs. arXiv preprint arXiv:260616988

opencodeai (2026) opencode cli (v1.19.0) agent harness. https://opencode.ai, gLM-5.3-Flash served via a Volcano-Engine-compatible gateway

Ren J, Wu S, Li Y, Zhu K, Xu S, Feng B, Yuan R, Zhang W, Batista-Navarro R, Yang J, Lin C (2026) Taco: A self-evolving framework for efficient terminal agents via observational context compression. arXiv preprint arXiv:260419572

Satish R, Sinha P, Kawada A, Yadwadkar NJ (2026) Beyond token savings: A systematic study of context compression in llm agents. arXiv preprint arXiv:260932961 Approx. 35,000 agent runs; SWE-bench Verified and Terminal-Bench 1.0

Shi J, Chen L (2026) Paritok-4b: Intent-conditioned context compression for coding agents. arXiv preprint arXiv:260824188

TASCO Research Group (2026) What is actually compressible? an empirical study of context compression opportunities in coding-agent trajectories (paper a, companion measurement study). Internal frozen manuscript and evidence chain (hash-frozen artifact index), 35,000+20,000+35,334 trajectories; 15-shape taxonomy; dual-ledger coverage

Thoughtworks (2026) Agentic coding trajectories. https://huggingface.co/datasets/thoughtworks/agentic-coding-trajectories, 15,000 coding-agent trajectories; manifest version 2026-05-02

Wang M, Luo F, Wang B, Yao G, Guo Y, Ning C, Chen H, Ma Y, Han J (2026a) When can agents forget their reasoning? iclr for long-horizon agent context compression. arXiv preprint arXiv:260929875

Wang X, Li B, Song Y, Xu FF, Tang X, Zhuge M, Pan J, Yue Y, Li H, et al. (2024) Openhands: An open platform for ai software developers as generalist agents. arXiv preprint arXiv:240716741

Wang Y, Shi Y, Yang M, Zhang R, He S, Lian H, Chen Y, Ye S, Cai K, Gu X (2026b) Swe-pruner: Self-adaptive context pruning for coding agents. arXiv preprint arXiv:260116746

Wu X, Li K, Zhao Y, Zhang L, Ou L, Yin H, Zhang Z, Jiang Y, Xie P, Huang F, Cheng M, Wang S, Cheng H (2025) Resum: Unlocking long-horizon search intelligence via context summarization. arXiv preprint arXiv:250913313

Xiao YA, Gao P, Peng C, Xiong Y (2025) Reducing cost of llm agents with trajectory reduction. arXiv preprint arXiv:250923586 AgentDiet; accepted at FSE 2026 (v2, Mar 2026)

Xue Y (2026) When and how context rot appears in coding agents: A white-box study of agent skills in code auditing. arXiv preprint arXiv:260717937

Yang J, Jimenez CE, Wettig A, Lieret K, Yao S, Narasimhan K, Press O (2024) Swe-agent: Agent-computer interfaces enable automated software engineering. arXiv preprint arXiv:240515793 NeurIPS 2024

Yang X, Chatterjee R, Udupi H, Ramavarapu V, Zhai C (2026) Skip the compressor: Coding agents fix more issues when they pick whole files. arXiv preprint arXiv:261021073

Ye R, Zhang Z, Li K, Yin H, Tao Z, Zhao Y, Su L, Zhang L, Qiao Z, Wang X, Xie P, Huang F, Chen S, Zhou J, Jiang Y (2025) Agentfold: Long-horizon web agents with proactive context management. arXiv preprint arXiv:251024699

Zeng Z, Li Y, Xie R, Ye W, Zhang S (2026) Attncompress: Dynamic attention-guided trajectory compression for software engineering agents. In: Proceedings of the 2026 ACM SIGPLAN International Symposium on Software Testing and Analysis (ISSTA), arXiv:2609.08318

Zhao X, Li H, Li S, Zhao T, Barr ET, Sarro F, Ye H (2026) Failure as a process: An anatomy of cli coding agent trajectories. In: arXiv preprint arXiv:2607.09510, 3,843 trajectories; 7 models; 3 scaffolds; 14 findings

Zheng M, OBrien D, Cui S, Pashakhanloo P, Mukherjee R, Kim M, Kuhar S (2026) Corvus: Context optimization for reliable versioned updates in software engineering agents. arXiv preprint arXiv:260722711